

Mastering Explainable AI (XAI)

Local and Global Interpretability with Feature Importance, SHAP, and LIME

Dr Marcelo Feighelstein

The Core Challenge: Why XAI Matters

High accuracy alone is insufficient for responsible deployment. Interpretability is the bridge between model outputs and human accountability.

🔍 Trust & Validation

Debug models to confirm they learn genuine signal — not spurious correlations tied to irrelevant features or data artifacts.

⚖️ Regulatory Compliance

GDPR's Article 22 enshrines a **"right to explanation"** for automated decisions — making interpretability a legal necessity, not a luxury.

🛡️ Fair & Ethical AI

Proactively surface bias against protected attributes — gender, race, age — before models reach production and cause real-world harm.



Taxonomy of Interpretability

XAI methods differ along two fundamental axes. Understanding these dimensions guides method selection for any given problem.



Global vs. Local

Global explanations describe overall model behavior across all inputs. **Local** explanations isolate the reasoning behind a single prediction.

Model-Specific vs. Model-Agnostic

Model-specific methods exploit internal architecture (e.g., tree splits). **Model-agnostic** methods treat any model as a black box — maximizing flexibility.

SHAP and LIME are both local *and* model-agnostic — making them universally applicable.

Traditional Approach: Global Feature Importance

Tree-based ensembles compute **Mean Decrease in Impurity (MDI)** — measuring how much each feature reduces weighted Gini impurity across all splits.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
import pandas as pd

# Load dataset and fit an ensemble model
data = load_breast_cancer()
X = pd.DataFrame(data.data,
                 columns=data.feature_names)
y = data.target

# Tip: random_state ensures reproducibility
#      n_estimators=100 is a reliable default
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
).fit(X, y)

# Tip: feature_importances_ gives a quick
#      global perspective but hides
#      local outliers and interactions
importances = pd.Series(
    model.feature_importances_,
    index=data.feature_names
)
importances.nlargest(10).plot(kind='barh')
```

Key Properties of MDI

→ Fast & Native

Computed at no extra cost during training — directly accessible via `feature_importances_`.

→ Global Scope

Summarizes behavior across all trees and all samples — ideal for initial feature screening.

→ Known Limitations

Biased toward high-cardinality features; cannot explain individual predictions.

Limitations of MDI: Permutation Importance

MDI is fast but flawed. **Permutation Importance** (PI) offers a more reliable alternative by directly measuring performance degradation when a feature's values are randomly shuffled.

MDI vs. Permutation Importance

MDI

Biased toward high-cardinality features. Computed on training data — can overfit.

Permutation

Model-agnostic. Evaluated on held-out test data. Reflects true generalization impact.

```
from sklearn.inspection import (
 permutation_importance
)

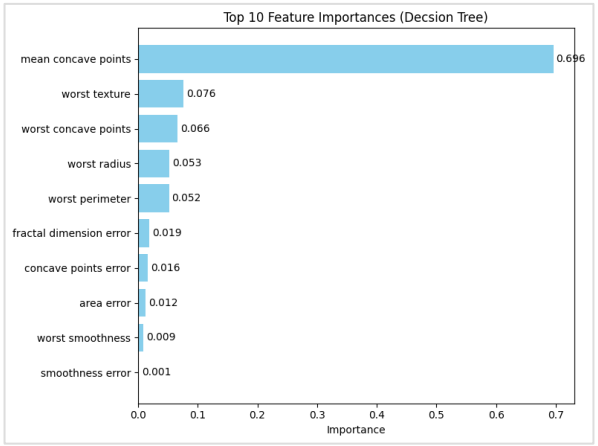
# Tip: always evaluate on the test set
# to avoid inflated importance scores
result = permutation_importance(
 model, X_test, y_test,
 n_repeats=30, # more = stabler est.
 random_state=42
)

pi = pd.Series(
 result.importances_mean,
 index=data.feature_names
)
pi.nlargest(10).plot(kind='barh')
```

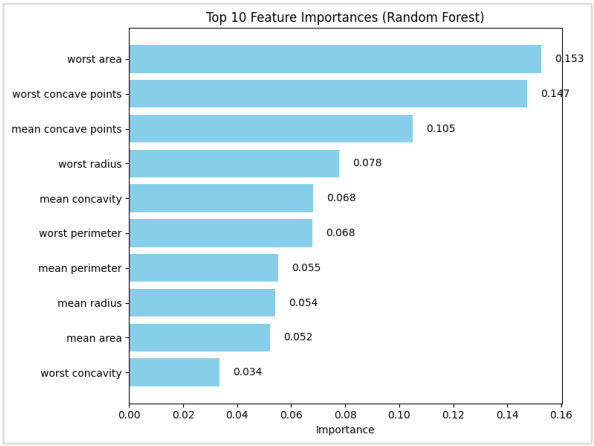
NOTEBOOK EXAMPLE

Notebook in Practice: MDI Feature Importance

EXPLANABILITY_USING_MOST_IMPORTANT_FEATURES_AND_SHAP.ipynb — Breast Cancer dataset, Decision Tree vs. Random Forest.



Decision Tree — Top 10 Features



Random Forest — Top 10 Features

```
importances = best_rf_model.feature_importances_
feat_imp = pd.Series(importances,
                     index=X.columns)
feat_imp.nlargest(10).plot(kind='barh')
```

Insight: Both models agree that “worst” area/perimeter and concave-point measurements dominate — but the exact ranking shifts between models, exactly the MDI instability the theory warns about.



Introducing SHAP: Game-Theoretic Explanations

SHAP (SHapley Additive exPlanations) grounds feature attribution in **cooperative game theory**. Each feature is a "player" contributing to the final prediction — its Shapley value is the fair, averaged marginal contribution across all possible feature coalitions.

Formally Grounded

Satisfies efficiency, symmetry, dummy, and additivity axioms — the only method with this guarantee.

Local & Global

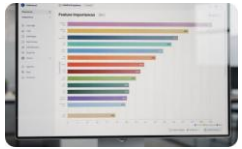
SHAP values explain individual predictions *and* aggregate into reliable global importance rankings.

Model-Agnostic

Works with any sklearn, XGBoost, LightGBM, or deep learning model via specialized explainers.

SHAP Visualization Gallery

SHAP provides a rich suite of plots, each designed to answer a different interpretability question — from global ranking to individual predictions.



Summary Bar Plot

Ranks features by mean absolute SHAP value — the most reliable global importance measure.



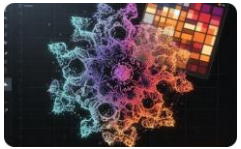
Beeswarm Plot

Overlays local SHAP values for all samples, revealing distribution, direction, and outliers simultaneously.



Waterfall / Force Plot

Explains a single prediction step-by-step — essential for case-by-case model auditing.



Dependence Plot

Shows how SHAP values change with feature magnitude across the dataset, with color encoding the feature's value.

SHAP in Practice: TreeExplainer

`TreeExplainer` is the fastest and most exact SHAP variant, purpose-built for tree-based models. It runs in polynomial time rather than the exponential time of naive Shapley computation.

```
import shap

# Tip: TreeExplainer is orders of magnitude
#      faster than KernelExplainer for trees
explainer = shap.TreeExplainer(model)

# shap_values shape: [n_samples, n_features]
# For classifiers: list of arrays (one per class)
shap_values = explainer.shap_values(X_test)

# --- Global summary plot ---
# Tip: plot=True shows feature direction (pos/neg)
shap.summary_plot(
    shap_values[1],  # class 1 (malignant)
    X_test,
    plot_type="bar"
)

# --- Single-prediction waterfall ---
# Tip: force_plot reveals per-sample logic
shap.force_plot(
    explainer.expected_value[1],
    shap_values[1][0],
    X_test.iloc[0]
)
```

Interpreting SHAP Output

1 Base Value

The model's average prediction over the training set — the starting point of every explanation.

2 Positive SHAP Values

Features that *push the prediction higher* than the base — toward the positive class.

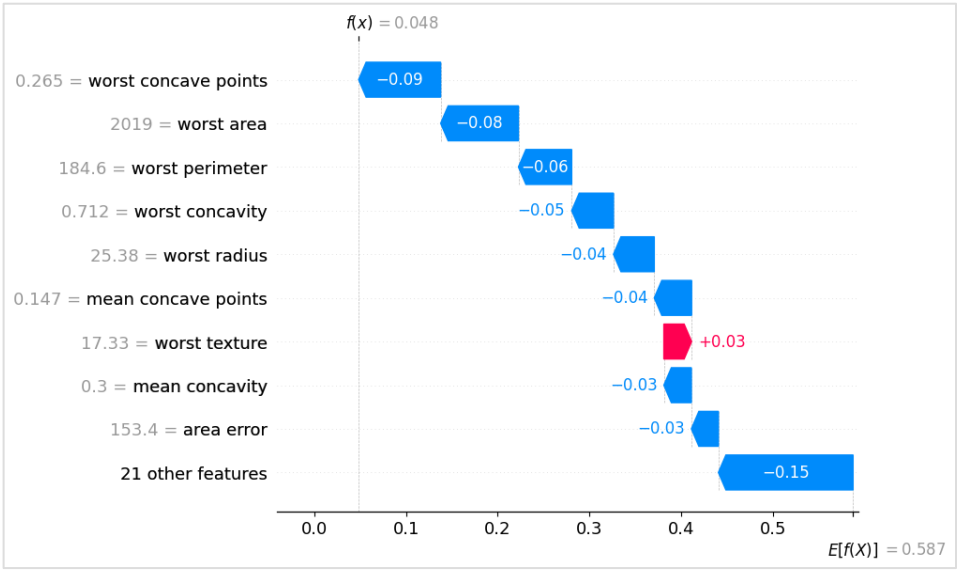
3 Negative SHAP Values

Features that *reduce the prediction* — evidence against the positive class.

NOTEBOOK EXAMPLE

Notebook in Practice: SHAP Local Explanation

EXPLANABILITY_USING_MOST_IMPORTANT_FEATURES_AND_SHAP.ipynb — explaining one prediction (local explainability).



How it's calculated: SHAP treats every feature as a player joining a coalition. For each possible ordering of features, it measures how much adding a feature shifts the prediction. Averaging that shift across all orderings gives the feature's Shapley value — its fair share of the gap between the baseline E[f(x)] and the final prediction f(x).

Insight: f(x) = 0.048 vs. baseline E[f(x)] = 0.587: worst concave points, worst area, and worst perimeter pull this patient firmly toward benign, while worst texture pushes slightly the other way.

Global Explanations via SHAP Aggregation

SHAP bridges local and global interpretability: by aggregating individual SHAP values across the entire dataset, we obtain the most theoretically sound global feature importance ranking available.

```
import numpy as np
import pandas as pd
import shap

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# --- Global importance: mean |SHAP| ---
# Tip: always use absolute values for ranking
#      signed means can cancel out and mislead
global_shap = pd.Series(
    np.abs(shap_values[1]).mean(axis=0),
    index=X_test.columns
).sort_values(ascending=False)

print(global_shap.head(10))

# --- Full beeswarm for distribution context ---
# Tip: beeswarm reveals not just rank but
#      direction and heterogeneity of effect
shap.summary_plot(
    shap_values[1],
    X_test
)
```

Global SHAP vs. MDI

Unbiased

Unlike MDI, SHAP global importance is not inflated by high-cardinality or correlated features.

Directional Insight

Beeswarm plots reveal whether high feature values push predictions up or down globally.

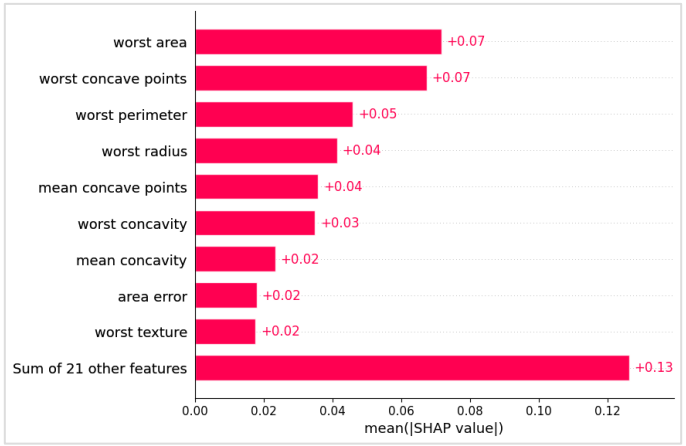
Additivity Guarantee

SHAP values always sum to the prediction — a mathematical consistency MDI cannot provide.

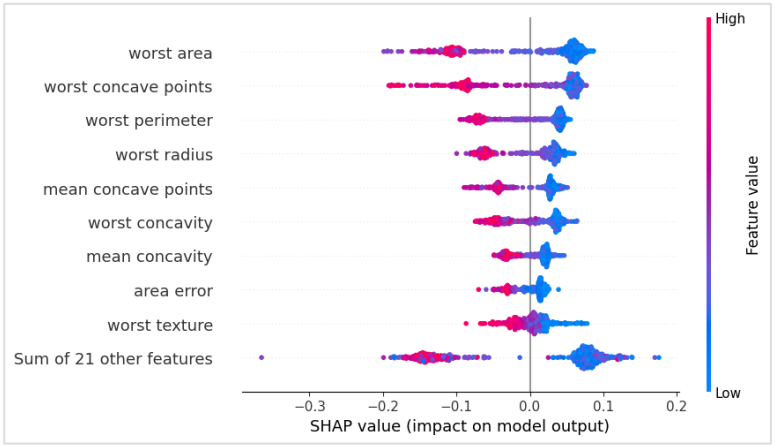
NOTEBOOK EXAMPLE

Notebook in Practice: SHAP Global Visualizations

EXPLANABILITY_USING_MOST_IMPORTANT_FEATURES_AND_SHAP.ipynb — explaining all predictions (global explainability).



Global Bar Plot — mean(|SHAP value|)



Beeswarm Plot — distribution & direction

Reading the beeswarm: Each dot is one patient. Horizontal position = that feature's SHAP value (impact on the prediction). Color = the feature's value for that patient (red = high, blue = low). Rows are ordered by mean |SHAP value|, so the top rows drive predictions the most.

Insight: worst area and worst concave points top both views; the beeswarm shows high values (red) push predictions toward malignant, confirming the direction MDI alone cannot show.

SHAP for Non-Tree Models: KernelExplainer

When the model is not tree-based — e.g., logistic regression, SVMs, or neural networks — `KernelExplainer` provides a model-agnostic SHAP solution using weighted linear regression over perturbed inputs.

```
from sklearn.linear_model import (
    LogisticRegression
)
import shap

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)

# Tip: summarize background with kmeans
#      to keep computation tractable
background = shap.kmeans(X_train, k=50)

# KernelExplainer treats model as black box
ke = shap.KernelExplainer(
    lr.predict_proba,
    background
)

# Tip: nsamples controls accuracy vs speed
#      default 2*X.shape[1]+2048 is often slow
shap_vals = ke.shap_values(
    X_test[:50],
    nsamples=500
)
```

KernelExplainer Trade-offs

Aspect	TreeExplainer	KernelExplainer
Speed	⚡ Fast	🐢 Slow
Exactness	Exact	Approximate
Model scope	Trees only	Any model
Recommended use	RF, XGB, LGB	LR, SVM, NN

Introducing LIME: Local Interpretable Model-Agnostic Explanations

LIME explains any classifier by fitting a simple, interpretable surrogate model — typically a sparse linear model — in the **local neighborhood** of the instance being explained. It perturbs the input, queries the black-box model, and learns a locally faithful approximation.

01

Perturb the Input

Generate synthetic samples by randomly masking or perturbing features around the instance of interest.

02

Query the Black Box

Obtain predictions from the original model for all perturbed samples — no model internals required.

03

Weight by Proximity

Assign higher weights to perturbed samples that are closer to the original instance using an exponential kernel.

04

Fit Surrogate Model

Train a sparse linear model on the weighted samples. The coefficients are the local explanation.

LIME in Practice: Tabular Data

The `lime` library provides a straightforward API for tabular, text, and image data. For tabular data, `LimeTabularExplainer` is the primary entry point.

```
from lime.lime_tabular import (
    LimeTabularExplainer
)
import numpy as np

# Tip: set discretize_continuous=True for
#      numeric features to improve readability
explainer = LimeTabularExplainer(
    training_data=X_train.values,
    feature_names=X_train.columns.tolist(),
    class_names=['benign', 'malignant'],
    mode='classification',
    discretize_continuous=True,
    random_state=42
)

# Explain a single instance
# Tip: num_features controls explanation
#      sparsity — 8-10 is a good default
exp = explainer.explain_instance(
    data_row=X_test.values[0],
    predict_fn=model.predict_proba,
    num_features=10
)

# Render interactive HTML visualization
exp.show_in_notebook(show_table=True)
```

Reading a LIME Explanation

Green Bars

Features whose values *support* the predicted class — positive local contribution.

Red Bars

Features whose values *contradict* the predicted class — negative local contribution.

Bar Length

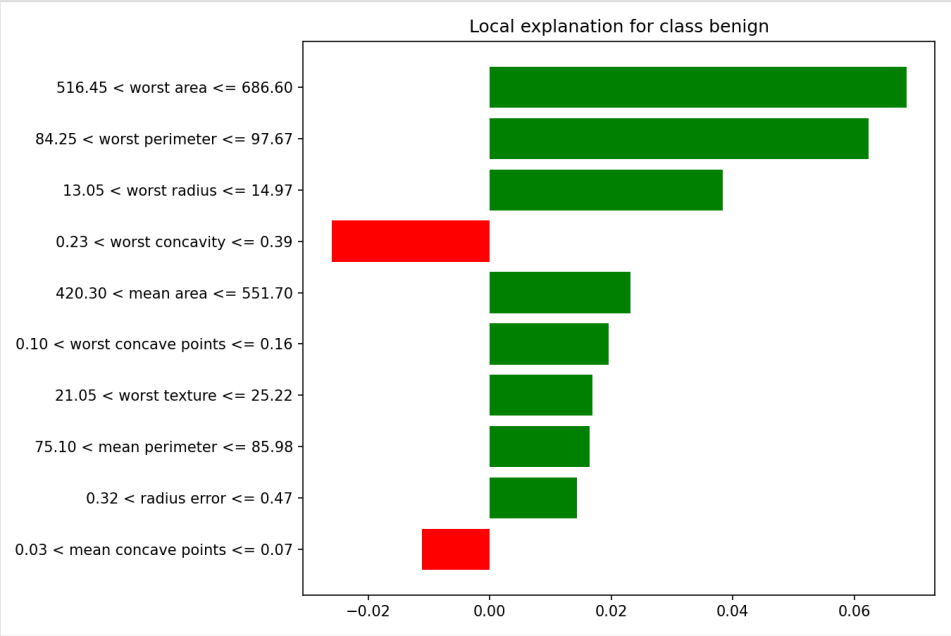
Magnitude of the local linear coefficient — longer bars signal stronger local influence.

LIME explanations are **locally faithful** but not globally consistent — always interpret them in context of the specific instance.

NOTEBOOK EXAMPLE

Notebook in Practice: LIME Explains a Diagnosis

LIME - TABULAR.ipynb — LimeTabularExplainer on the optimal Random Forest model.

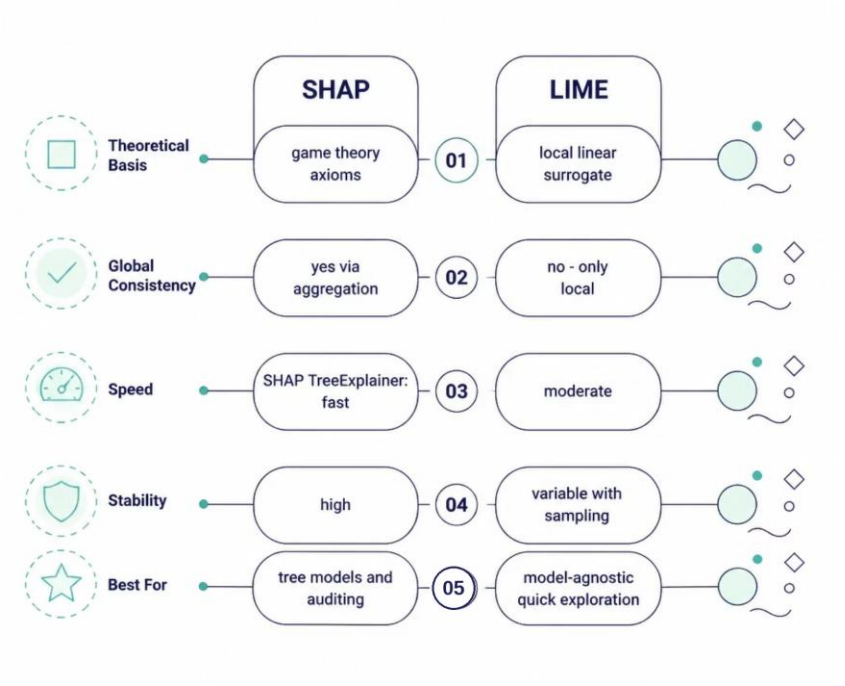


How it's calculated: LIME perturbs the instance by masking or discretizing its feature values, then asks the black-box model to predict each perturbed sample. Samples closer to the original instance get more weight via an exponential kernel. A sparse linear model is fit on these weighted samples — its coefficients are the green/red bars shown here.

Insight: Model predicts benign with 95.5% confidence. worst area > 516.45 is the strongest green (supporting) signal; worst concavity is the lone red feature arguing for malignant.

SHAP vs. LIME: A Head-to-Head Comparison

Both methods explain individual predictions, but they differ fundamentally in theoretical grounding, consistency, and computational cost. Choosing correctly depends on your model type and use case.



Practical Decision Guide

→ Use SHAP when...

You need theoretically guaranteed, globally coherent explanations — especially with tree-based models in regulated environments.

→ Use LIME when...

You need quick, intuitive local explanations for *any* model type, including deep learning, without installing specialized libraries.

→ Use both when...

Cross-validating explanations — if SHAP and LIME agree on a prediction, confidence in the explanation increases substantially.

Bias Detection with SHAP

SHAP is a powerful auditing tool: by slicing SHAP values along sensitive attributes such as gender, race, or age, practitioners can quantify whether a model is making decisions for **the right reasons**.

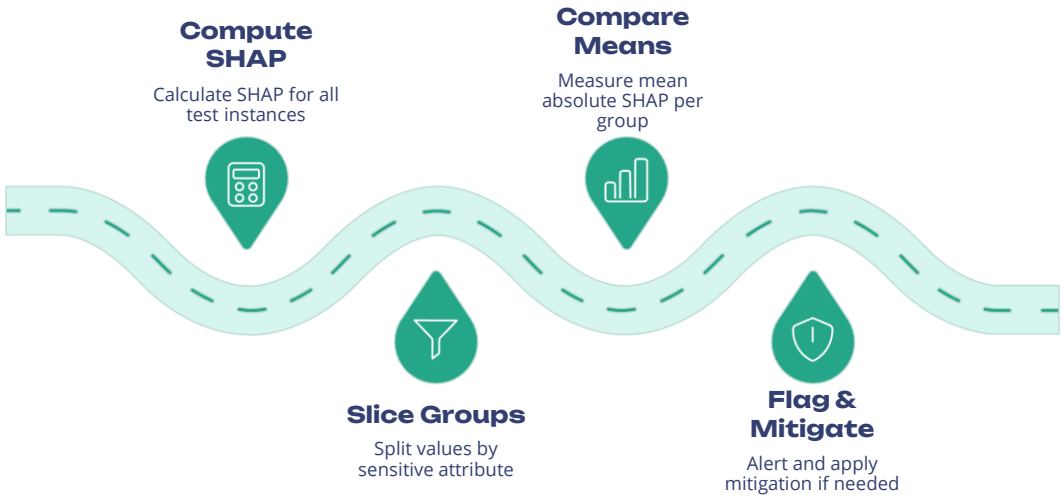
```
import pandas as pd
import shap

# Assume X_test has a 'gender' column
# and shap_values[1] is the positive class
shap_df = pd.DataFrame(
    shap_values[1],
    columns=X_test.columns
)
shap_df['gender'] = X_test['gender'].values

# Tip: compare mean SHAP by group to flag
#      disparate impact on protected classes
group_shap =
shap_df.groupby('gender').mean()
print(group_shap[['income',
'credit_score']])

# Tip: if a protected attribute like gender
#      has a large non-zero SHAP value,
#      the model may encode direct bias
print(shap_df.groupby('gender')[
    'gender'
].mean())  # Should be ~0 for fairness
```

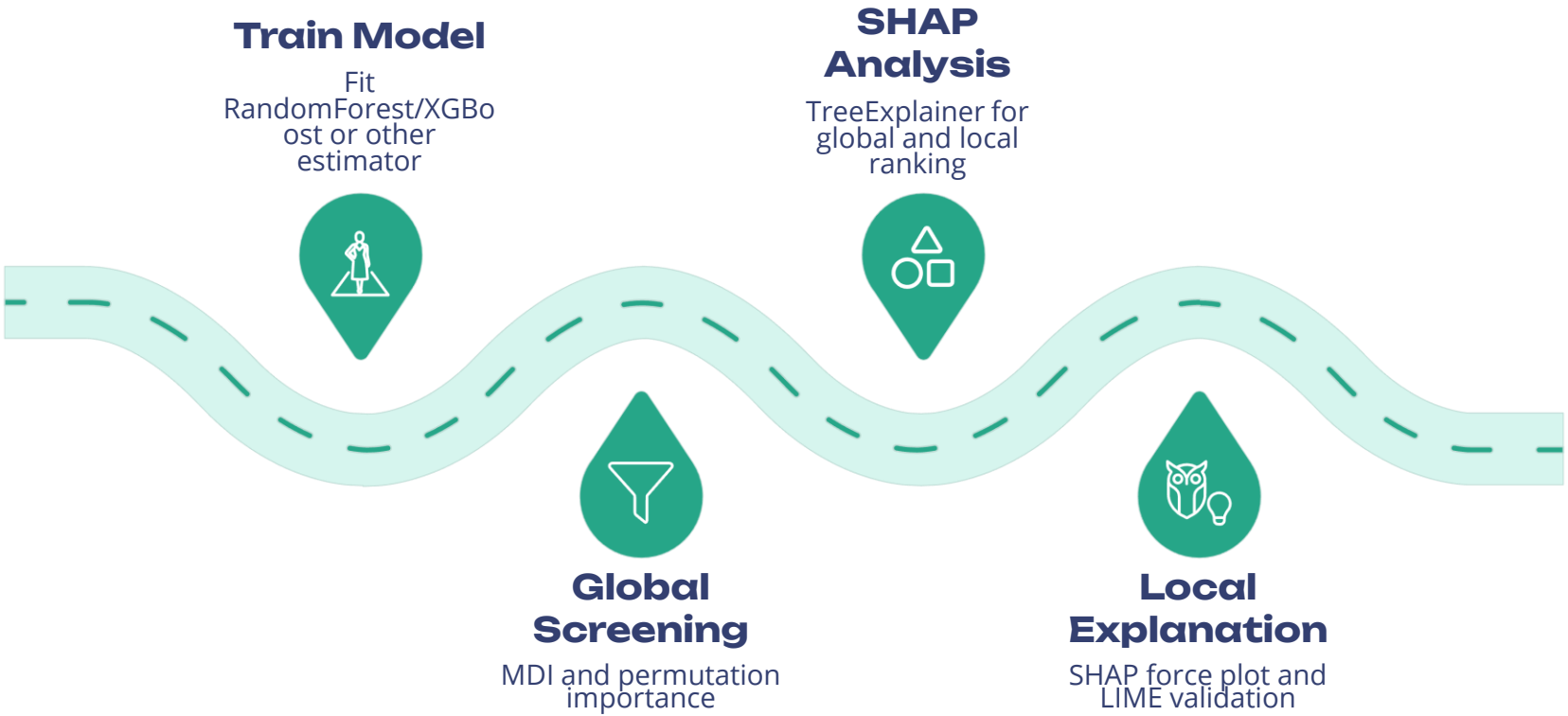
SHAP Fairness Audit Workflow



If a protected attribute carries non-negligible SHAP values, the model is directly using it — requiring feature removal, re-weighting, or adversarial debiasing before deployment.

Unified XAI Pipeline: Putting It All Together

A robust XAI workflow integrates multiple complementary methods — from traditional importance to SHAP and LIME — across both local and global scopes.



Step 1–2: Screen
Use MDI and permutation importance for fast initial feature selection and sanity checks.

Step 3–4: Explain
SHAP for rigorous global and local analysis; LIME as an independent cross-validation lens.

Step 5: Deploy
Log explanations, monitor drift, and translate SHAP values into regulatory reason codes.

Key Takeaways & Next Steps

1

Start with MDI, graduate to SHAP

MDI is a fast first pass. SHAP delivers theoretically grounded, globally consistent, and locally precise explanations.

2

LIME cross-validates local reasoning

Use LIME as an independent sanity check — agreement between SHAP and LIME substantially increases explanation confidence.

3

Explainability is a systems problem

Log SHAP values in production, monitor explanation drift, and translate outputs into regulatory-compliant reason codes.

4

Next: Deep Learning XAI

Explore `shap.DeepExplainer`, integrated gradients, and attention visualization for transformer-based architectures.

Lab Assignment: Apply the full XAI pipeline — MDI → Permutation → SHAP → LIME — to the UCI Adult Income dataset. Audit the model for gender and race bias using SHAP slicing before the next lecture.



Thank You!

Go explore data — get your hands dirty, break things, and learn by doing.

Email

MarcelFei@telhai.ac.il

Office Hours

Per Request

Location

Building 6, Room 309, Eastern Campus